

Implementation Notes

Speech Understanding — Programming Assignment 2

One Non-Obvious Design Choice per Task

Roll No: M25CSA028 April 2026

<https://github.com/kingkenche/Speech-Understanding/tree/Assignment-2>

Task 1.1 — Multi-Head LID: Frame-Level CNN over Utterance-Level Classification

Design Choice: The LID model (`FrameLIDNet`) operates on 800 ms sliding windows (40×20 ms frames) with a stride of one frame, rather than classifying whole utterances.

Rationale: Code-switching in Hinglish happens sub-utterance — often mid-sentence (e.g., “yeh *stochastic* process hai”). Utterance-level classifiers average out these transitions, producing a single language label that is wrong for both segments. By sliding a CNN over log-mel frames with a receptive field of ~ 800 ms, the model produces a per-frame language probability sequence. Switch boundaries are detected as the first frame where the predicted class changes, achieving the ± 200 ms precision required by the assignment. A multi-head variant applies two separate linear classification heads to the shared encoder — one per language — enabling independent calibration of English and Hindi posteriors before the argmax decision.

Task 1.2 — Constrained Decoding: Additive Logit Bias over Hard Vocabulary Constraints

Design Choice: The `NGramLogitBiasProcessor` adds a *scaled n -gram probability* to each candidate token’s logit rather than zeroing out non-technical tokens (hard constraint / vocabulary whitelisting).

Rationale: Hard vocabulary constraints force Whisper to pick technical terms even when acoustic evidence strongly suggests a common word, causing hallucinations. Additive biasing

$$\tilde{z}_t^{(v)} = z_t^{(v)} + \alpha \cdot \log P_{\text{ngram}}(x_v \mid h)$$

instead *probabilistically steers* the model: if the n -gram model assigns high probability to “cepstrum” given the context “mel frequency”, the bias nudges beam-search ranking upward without overriding acoustic evidence. The scale factor $\alpha = 4.0$ was chosen so that a trigram probability of 0.1 contributes approximately the log-space equivalent of one acoustic frame of evidence — a sweet spot that boosts recall of technical terms without degrading fluency.

Task 2.1 — Hinglish G2P: Greedy Digraph Matching over Neural Lookup Tables

Design Choice: The `_word_to_ipa()` function uses greedy left-to-right *digraph matching* rather than a trained G2P neural model for the Devanagari-romanized portion of Hinglish.

Rationale: No publicly available G2P model handles code-switched Romanized Hindi reliably.

Hindi romanization is not standardized — “dh”, “bh”, “ph” are aspirated stops mapping to different IPA symbols than their component characters. A greedy digraph scan (check 2-character pair first, fall back to single character) reliably handles all standard ITRANS-style romanizations without any training data. For English tokens, standard grapheme-to-IPA mapping handles the ASCII portion. The combined system correctly converts “bhasha” → /bh-aa-sh-a/ and “deep” → /d-ii-p/ without any trained model, making it fully reproducible with zero labelled data.

Task 3.2 — Prosody Warping: Energy-Scaled DTW over Vocoder Pitch Shifting

Design Choice: Instead of vocoder-based pitch shifting (PSOLA), `apply_prosody_warping()` applies DTW-aligned F_0 and energy ratios directly to the waveform amplitude envelope.

Rationale: Vocoder-based pitch shifting requires resynthesis from acoustic parameters, introducing a full vocoder round-trip that degrades perceptual quality. The surrogate approach — warping the energy envelope via DTW alignment indices and applying the F_0 ratio as a mild amplitude modulation — transfers the *teaching rhythm* (louder emphasis on key words, pauses between concepts) without altering the fundamental perceptual character of the synthesized voice. For TTS output, the energy envelope encodes prosodic emphasis more reliably than raw F_0 because synthesized F_0 is already smooth; modulating it again introduces artifacts. This avoids the MOS degradation associated with double-synthesis while still preserving the temporal pattern of the professor’s lecture delivery.

Task 4.1 — Anti-Spoofing: Shallow TDNN over SVM on LFCC Features

Design Choice: The LFCCTDNN model uses two 1D convolutional layers (equivalent to a shallow Time Delay Neural Network) with adaptive average pooling rather than a linear SVM.

Rationale: LFCC feature sequences vary in length across utterances; an SVM requires fixed-size input. Padding to maximum length and flattening loses temporal structure and creates a high-dimensional sparse input where most classifiers fail. The TDNN architecture (`Conv1D` → `ReLU` → `Conv1D` → `AdaptiveAvgPool1d(1)`) naturally handles variable-length LFCC matrices by collapsing the time axis to a single 96-dimensional vector via global average pooling, which a linear head then classifies. This is simpler than attention and achieves comparable EER to SVM on the ASVspoof benchmark for the binary bona-fide vs. spoof case, while remaining fully differentiable for FGSM gradient-based adversarial analysis in Task 4.2.